

$4 \leq N, M \leq 10$ 이므로 포탑 배열 Turret turrets[101];

$N \times M$ 격자가 있고, 모든 위치에는 포탑이 존재합니다. (즉, 포탑의 개수는 NM 개)

각 포탑에는 공격력이 존재하며, 상황에 따라 공격력이 줄어들거나 늘어날 수 있습니다. 또한, 공격력이 0 이하가 된다면, 해당 포탑은 부서지며 더 이상의 공격을 할 수 없습니다. 최초에 공격력이 0인 포탑 즉, 부서진 포탑이 존재할 수 있습니다. 공격력 = 체력

아래와 같은 4×4 크기의 격자를 생각해보겠습니다. (1, 1), (2, 2), (3, 2) 를 포함한 총 7개의 칸은 이미 부서진 포탑을 의미합니다.

0	1	4	4
8	0	10	13
8	0	11	26
0	0	0	0

int board[11][11];
Turret turrets[101];
또는
Turret turrets[11][11];
방법론 고찰.
(깃허브 cpp 코드 참고).

4가지 동작 는 말하면 거.

하나의 턴은 다음의 4가지 액션을 순서대로 수행하며, 총 K 번 반복됩니다. 만약 부서지지 않은 포탑이 1개가 된다면 그 즉시 중지됩니다.

1. 공격자 선정 1번 동작

1-1번 동작 1-2번 동작

부서지지 않은 포탑 중 가장 약한 포탑이 공격자로 선정됩니다. 공격자로 선정되면 가장 약한 포탑이므로, 핸디캡이 적용되어 $N + M$ 만큼의 공격력이 증가됩니다.

1-3번 동작

가장 약한 포탑은 다음의 기준으로 선정됩니다. 조건 4가지 → 모두 대소 비교 조건이므로, ☆

struct에서 operator < 함수 생성

1. 공격력이 가장 낮은 포탑이 가장 약한 포탑입니다.
2. 만약 공격력이 가장 낮은 포탑이 2개 이상이라면, 가장 최근에 공격한 포탑이 가장 약한 포탑입니다. (모든 포탑은 시점 0에 모두 공격한 경험이 있다고 가정하겠습니다.)
3. 만약 그러한 포탑이 2개 이상이라면, 각 포탑 위치의 행과 열의 합이 가장 큰 포탑이 가장 약한 포탑입니다.
4. 만약 그러한 포탑이 2개 이상이라면, 각 포탑 위치의 열 값이 가장 큰 포탑이 가장 약한 포탑입니다.

위의 예시에서 공격자를 선정해보면, 가장 낮은 공격력은 1이기 때문에, 아래 그림과 같이 (1, 2) 위치에 있는 포탑이 공격자로 선정되며, 공격력이 $8(= 4(N) + 4(M))$ 만큼 증가하여 9가 됩니다.

0	+8 1 $\Rightarrow$ 9	4	4
8	0	10	13
8	0	11	26
0	0	0	0

2. 공격자의 공격 2번 동작 레이저 공격 포탄 공격 2-1번 동작. 가장 강한 포탑 선정

위에서 선정된 공격자는 자신을 제외한 **가장 강한 포탑**을 공격합니다.

가장 강한 포탑은 위에서 정한 **가장 약한 포탑 선정 기준의 반대**이며, 다음과 같습니다.

1. **공격력이 가장 높은** 포탑이 가장 강한 포탑입니다. ↳ 따로 만들 필요 X.
2. 만약 공격력이 가장 높은 포탑이 2개 이상이라면, **공격한지 가장 오래된 포탑**이 가장 강한 포탑입니다. (모든 포탑은 시점 0에 모두 공격한 경험이 있다고 가정하겠습니다.)
3. 만약 그러한 포탑이 2개 이상이라면, 각 포탑 위치의 **행과 열의 합이 가장 작은** 포탑이 가장 강한 포탑입니다.
4. 만약 그러한 포탑이 2개 이상이라면, 각 포탑 위치의 **열 값이 가장 작은** 포탑이 가장 강한 포탑입니다.

위의 예시에서 공격 대상자를 선정해보겠습니다. 공격 대상인 가장 강한 포탑은 공격력이 가장 큰 26의 공격력을 가진 (3, 4) 위치의 포탑이 됩니다.

1번 동작 때 만든 operator <로 정렬하면,

turret 배열

			...			
--	--	--	-----	--	--	--

← 가장 약한
가장 강한 →

⇒ ∴ 가장 약한 건 인덱스 0,
가장 강한 건 인덱스 N·M.
(단, 체력 0 제외).

0	9	4	4
8	0	10	13
8	0	11	26
0	0	0	0

공격을 할 때에는 **레이저 공격**을 먼저 시도하고, 만약 그게 안 된다면 **포탄 공격**을 합니다. 각 공격의 규칙은 다음과 같습니다.

(1) 레이저 공격 2-2 번 동작

레이저는 다음의 규칙으로 움직입니다.

1. 상하좌우의 4개의 방향으로 움직일 수 있습니다. **조건**.
2. 부서진 포탑이 있는 위치는 지날 수 없습니다.
3. 가장자리에서 막힌 방향으로 진행하고자 한다면, 반대편으로 나옵니다. (예를 들어, 위의 예시에서 (2, 3)에서 오른쪽으로 두 번 이동한다면, (2, 3) → (2, 4) → (2, 1) 순으로 이동합니다.)

→ BFS (행동 강경) 2-2-1번 공격

레이저 공격은 공격자의 위치에서 공격 대상 포탑까지의 최단 경로로 공격합니다. 만약 그러한 경로가 존재하지 않는다면 (2) 포탄 공격을 진행합니다. 만약 경로의 길이가 똑같은 최단 경로가 2개 이상이라면, 우/하/좌/상의 우선순위대로 먼저 움직인 경로가 선택됩니다.

dx/dy 배열 원소 순서 이대로 설정

최단 경로가 정해졌으면, 공격 대상에는 공격자의 공격력 만큼의 피해를 입히며, 피해를 입은 포탑은 해당 수치만큼 공격력이 줄어듭니다. 또한 공격 대상을 제외한 레이저 경로에 있는 포탑도 공격을 받게 되는데, 이 포탑은 공격자 공격력의 절반 만큼의 공격을 받습니다. (절반이라 함은 공격력을 2로 나눈 몫을 의미합니다.)

why? BFS는 경로를 저장하지는 않음.

(2) 포탄 공격 2-3 번 동작

2-3-1번 동작

공격 대상에 포탄을 던집니다. 공격 대상은 공격자 공격력 만큼의 피해를 받습니다. 추가로 주위 8개의 방향에 있는 포탑도 피해를 입는데, 공격자 공격력의 절반 만큼의 피해를 받습니다. (절반이라 함은 공격력을 2로 나눈 몫을 의미합니다.) 공격자는 해당 공

격에 영향을 받지 않습니다. 만약 가장자리에 포탄이 떨어졌다면, 위에서의 레이저 이동처럼 포탄의 추가 피해가 반대편 격자에 미치게 됩니다.

위의 예시에서 공격자가 공격 대상을 레이저로 공격하기 위한 최단 경로는 아래 그림과 같습니다. 최단 경로가 2개 이상 있지만, **오른쪽**으로 먼저 움직이는 것이 우선 순위가 높기 때문에, 다음의 경로가 선택된 것입니다. 최단 경로가 정해졌기 때문에, 공격 대상에는 9만큼의 피해를 입히고, 경로에 있는 포탑에는 $4(= 9/2)$ 만큼의 피해를 입힙니다.

0	9	-4 4 $\Rightarrow$ 0	-4 4 $\Rightarrow$ 0
8	0	10	-4 13 $\Rightarrow$ 9
8	0	11	-9 26 $\Rightarrow$ 17
0	0	0	0

포탑 구조체에 공격력이 0인 경우.

3. 포탑 부서짐 3번 동작

↪ C++에서 0은 False
0을 제외한 모든 정수는 True

공격을 받아 공격력이 0 이하가 된 포탑은 부서집니다.

⇒ `if(!공격력) continue;`
이해 필수.

4. 포탑 정비 4번 동작

4-1번 동작

공격이 끝났으면, 부서지지 않은 포탑 중 공격과 무관했던 포탑은 공격력이 1씩 올라갑니다. 공격과 무관하다는 뜻은 공격자도 아니고, 공격에 피해를 입은 포탑도 아니라는 뜻입니다.

위의 예시에서 공격과 무관했던 다음 4개의 포탑은 정비를 받아 공격력이 1씩 증가합니다. 이렇게 첫 번째 턴이 종료됩니다.

반드시 예제 설명 꼼꼼하게 읽고 이해.

0	9	0	0
+1 8 => 9	0	+1 10 => 11	9
+1 8 => 9	0	+1 11 => 12	17
0	0	0	0

다음 두 번째 턴이 시작되면, 다시 공격자 선정이 시작됩니다.

첫 번째 규칙에 따라 가장 공격력이 작은 9 만큼의 공격력을 가진 (1, 2), (2, 1), (2, 4), (3, 1) 위치의 포탑이 공격자 후보가 됩니다. 두 번째 규칙에 의해 가장 최근에 공격한 포탑인 (1, 2) 포탑이 가장 약한 포탑이므로 공격자로 선정됩니다. 마찬가지로 핸디캡으로 8 만큼의 공격력을 얻습니다.

0	+8 9 => 17	0	0
9	0	11	9
9	0	12	17
0	0	0	0

다음으로 공격 대상을 선택하면, 공격력이 가장 높은 (3, 4)가 선택됩니다. 공격자의 위치에서 공격 대상의 위치까지 이동하는 경로가 없기 때문에, 레이저 공격은 불가능하여 포탄 공격을 수행하게 됩니다. 포탄은 공격 대상인 (3, 4) 위치에 떨어지며, 공격 대상은 17만큼의 피해를 받고, 주변 8개의 방향에 있는 포탑은 8만큼의 피해를 받습니다.

0	17	0	0
-8 9 => 1	0	-8 11 => 3	-8 9 => 1
-8 9 => 1	0	-8 12 => 4	-17 17 => 0
0	0	0	0

마지막으로 공격과 무관했던 포탑은 정비를 받아 1 만큼의 공격력을 얻지만, 위의 예시에서는 부서지지 않은 포탑 중 공격과 무관했던 포탑이 없으므로, 최종 모습은 다음과 같습니다.

0	17	0	0
1	0	3	1
1	0	4	0
0	0	0	0

전체 과정이 종료된 후 남아있는 포탑 중 가장 강한 포탑의 공격력을 출력하는 프로그램을 작성해보세요.

입력

제한 조건

-
-
-
-

출력

입력 예제

예제 1

입력

4 4 1
0 1 4 4
8 0 10 13
8 0 11 26
0 0 0 0

출력

17

예제 2

입력

4 4 2
0 1 4 4
8 0 10 13
8 0 11 26
0 0 0 0

출력

17

제한